vsh

Language Specification

Nathan Tebbs · Draft · 2026-07-16

An interactive, compiled, C-like shell language. You type C, it compiles to native code, it runs. The compiler is its own.

Contents

1. About This Document

This is a sketch, not a standard. vsh is early enough that most of what follows is intention rather than behaviour, and the document's main job is to keep those two apart. Every section carries one of three marks:

BUILT	Built and exercised. The code does this today.
DECIDED	Decided, with a reason on record, but not written yet.
OPEN	Genuinely unsettled. Do not assume. These are the interesting parts.

Where a decision was made for a reason, the reason is written down. Reversing a decision is cheap; rediscovering why it was made is not.

The worklist lives in `TODO.md`. This document explains, that one enumerates.

2. Overview

2.1. What vsh Is

vsh is a statically typed, imperative, compiled language with a C-like feel, driven from a REPL, intended to become a shell you live in rather than only script from.

Typing an expression compiles it. There is no interpreter and no bytecode. The text is lexed, parsed, name-resolved, emitted as arm64 machine code, written into an executable page, and called as a function. The value comes back in a register.

2.2. Where It Comes From

Three inspirations, each contributing one thing:

bash	Pipes, process control, and the shell as a place you live in.
GHCi	The REPL loop. Type an expression, get a value, with a compiler catching mistakes before anything runs. The loop only. Functional programming is explicitly not a goal.
C	A manual memory model, no hidden runtime, direct to the metal, and the appeal of building your own tools rather than gluing other people's together.

TempleOS's HolyC is the closest existing thing: a shell whose REPL was a live C compiler. vsh takes that pitch and applies it deliberately, with answers to the things bash and raw C get wrong.

2.3. Non-Goals

- **Functional programming.** The REPL borrows from GHCi. The language does not.
- **A hidden runtime.** No garbage collector, no exceptions, no implicit allocation.
- **Embedding someone else's compiler.** Considered and rejected; see below.

2.4. On Not Using tinycc

The original plan embedded tinycc through `libtcc`. It is not used, for two independent reasons.

The first is the point of the project. Writing the compiler is the work. Embedding libtcc would put someone else's code exactly where vsh's belongs, and would turn the interesting problems, which are persistence, arena scoping, and compile granularity, into problems of fighting libtcc's model instead of designing one.

The second is mechanical. Homebrew deprecates tcc as unsupported upstream and disables it on 2026-09-16. Stable is 0.9.27, tagged 2017, three years before Apple Silicon shipped. It publishes no bottle for any platform, and its formula records build failures. It was a poor foundation independent of taste.

3. Execution Model

BUILT

3.1. The Pipeline

One line becomes one function.

```
lex  ->  parse  ->  check  ->  jit  ->  call
```

`lex` turns bytes into tokens. `parse` turns tokens into a tree. `check` resolves every name to a storage address. `jit` walks the tree emitting arm64 and installs the bytes in executable memory. `repl` calls the result and prints what comes back.

Data flows one way. No stage reaches backward.

3.2. Executable Memory

The backend needs pages it can both write and jump to. On Apple Silicon that is constrained, and the constraints were measured rather than assumed:

Question	Answer
Does `MAP_JIT` work unsigned, with no entitlement?	Yes
Can a live code page be patched and re-called?	Yes
Does reserve-then-commit compose with `MAP_JIT`?	Yes

Can ordinary reserved memory be made executable later?	No, `EACCES`
Is the write-protect toggle per-mapping?	No, per-thread

Two of these shaped the code. Because `MAP_JIT` must be passed at `mmap` time and cannot be added by `mprotect` afterward, executable memory is a separate reservation rather than a flag on the existing one. Because the write toggle is thread-wide, it cannot be modelled as a nestable scope: `jit.c` builds instructions into a scratch array and installs them in a single write rather than toggling as it walks the tree.

That a live page can be patched and re-called is the mechanism by which redefinition will eventually work.

3.3. Codegen

Codegen is a stack machine. Every node leaves its value on the machine stack, and a binary operator pops two and pushes one. Push and pop move the stack pointer by 16 rather than 8, because arm64 requires `sp` to stay 16-byte aligned.

This emits instructions a register allocator would not. It is correct at any expression depth, needs no allocation pass, and is easy to read. Registers come later, and `TODO.md` tracks it.

Integer literals load with `movz` and up to three `movk`. A negative value fills all four halfwords and so costs four instructions.

The backend has `cbz` and `b`, and patches a branch's offset once its target is known. Division by zero forced that machinery; control flow will reuse it.

3.4. Compile Granularity

OPEN

Today one input line is one function, compiled whole. HolyC compiled each line as its own function too. Whether vsh keeps that, or moves to explicit `{}`-delimited blocks, is unsettled and interacts with control flow and functions. It is deliberately not being answered before those exist.

4. Memory Model

BUILT

4.1. Two Arenas, Two Lifetimes

A REPL has exactly two lifetimes, so the session has exactly two arenas.

Session	Symbols, their names, and their storage. Never popped. Lives until the session ends.

Scratch	Tokens, tree, and instruction buffer. Popped after every line, because none of it means anything once the line has run.

Both are arenas from the base layer: they reserve a gigabyte of address space and commit pages only as the bump pointer reaches them. Reserving costs nothing until touched.

There is no `malloc` in vsh, no `free`, and no per-allocation lifetime. Raw manual allocation is unworkable in a REPL, where a variable declared on line 1 and read on line 50 makes "did I free this?" impossible to answer by hand. A garbage collector would betray the no-hidden-runtime goal. Arena scoping is the middle ground, and it is what the base layer already provides.

4.2. Why State Survives A Line

This is the problem the project is really about, so it is worth stating plainly.

A normal one-shot compile throws away all state when the process exits. A shell cannot. The answer is that **vsh owns the storage, not the compiler.**

`sym.c` gives every variable a slot in the session arena. Because that arena is never popped, the slot's address is fixed the moment the name is declared. Codegen then bakes the address in as an immediate and reaches the variable through a plain load of a constant address. A line compiled minutes later, as a separate function, in a different page, loads from the same address and sees the same value.

Nothing is shared with the compiler because the compiler owns nothing.

4.3. Slots Never Move

The symbol table hands out slot pointers, never `Symbol` pointers. The `Symbol` array is a dynamic array, so it moves when it grows, and a `Symbol *` taken before a growth would dangle. Slots are pushed individually and never move.

This is verified: declaring fifty variables forces roughly four reallocations of the array, and the first variable still reads correctly afterward.

Names are copied into the session arena at declaration. A token's name borrows the line buffer, and the next prompt overwrites it.

5. Lexical Structure

BUILT

Integer	One or more decimal digits. Value must fit `i64`; overflow is a diagnostic, never a wrap.
Name	`[A-Za-z_][A-Za-z0-9_]*`

Keyword	`i64`. The only one.
Operators	`+` `-` `*` `/` `=`
Punctuation	`;` `(` `)`
Whitespace	Space, tab, carriage return, newline. Separates tokens, otherwise ignored.
Comments	`//` to end of line, and `/* */`, which does not nest. A `/*` that never closes is a diagnostic rather than a silent run to end of file.

Whitespace and comments are skipped together, because both are only gaps between tokens. A lone `/` is division, so the two-character lookahead is what tells `10 / 2` from `10 // 2`.

Newlines are whitespace and nothing more. The grammar spans them, which is why a file needs no separate parser.

There are no string or character literals. A byte that begins no token is a diagnostic, and the lexer always advances past it, so a bad byte cannot loop the parser.

6. Grammar

BUILT

```
line    := stmt* expr?
stmt    := decl | expr ';' | ';'
decl    := 'i64' NAME '=' expr ';'
expr    := NAME '=' expr | addsub
addsub  := term (('+' | '-') term)*
term    := unary (('*' | '/') unary)*
unary   := '-' unary | primary
primary := INT | NAME | '(' expr ')'
```

Recursive descent, one token of lookahead. The first error wins and the rest of the parse unwinds without reporting, so a reader sees the cause rather than its consequences.

Trailing input is an error. Without that rule `1 2` would quietly evaluate to 1, which is the class of silent wrongness vsh exists to avoid.

6.1. What The Semicolon Is For

BUILT

A line is any number of statements, optionally ending in an expression with **no** semicolon. That trailing expression is the line's value and the only thing the prompt prints.

```
vsh> i64 x = 100; i64 y = 100; i64 xy = x * y;
vsh> xy
10000
vsh> i64 a = 5; a * 2
10
vsh> 2 + 3 * 4
14
vsh> 2 + 3 * 4;
vsh>
```

So the semicolon means "discard this value", which is exactly what it already means in C, where `foo();` throws away what `foo` returned. Until statements could be sequenced, the semicolon only terminated a declaration and carried no weight, which made it decoration. Now it earns its place.

This rule is also what makes files work. A file is a sequence of statements, all terminated, so loading one runs it and prints nothing. A prompt is the same grammar with a trailing expression on the end.

An empty statement is allowed, as in C, so a stray `;;` costs nothing.

Statements chain through a `next` pointer rather than nesting, and both the checker and codegen walk that chain with a loop. A file of ten thousand statements therefore costs one stack frame rather than ten thousand.

7. Types

BUILT for `i64`. OPEN for everything else.

`i64` is the only type. It is 64-bit, signed, two's complement.

The intent is that the fixed-width types the base layer already uses, `i8` through `u64`, `f32`, `f64`, and `b32`, surface as language types, and that `check.c` grows from a name resolver into a type checker to match. There is no schedule for this and no decision about implicit conversion, which is one of the things C gets wrong and is worth not repeating by reflex.

8. Statements and Expressions

8.1. Declaration

BUILT

```
i64 x = 5;
```

A declaration is a **statement**, so it prints nothing and does not move `it`.

The initialiser is resolved before the name is declared. That makes `i64 x = x + 1;` read the old `x`, and makes `i64 y = y;` an error rather than a read of storage that was created a moment earlier and never written.

8.2. Assignment

BUILT

```
x = 9
```

Assignment is an **expression**, as in C. It is the lowest precedence, binds to the right, and its value is the value assigned. So `a = b = 1` binds right, and `1 + (a = 7)` is 8. At the prompt it prints and moves `it`, because it is an expression and not a statement.

Only a name may sit left of `=`. The target is parsed as an ordinary operand and then rejected if it is not a name, because deciding on `=` needs the whole left side in hand first. `(a) = 2` is accepted, which is what C does with a parenthesised lvalue.

Assigning to an undeclared name is an error. Declaration is the only thing that creates a name.

8.3. Redeclaration

BUILT

Redeclaring a name makes a **new slot** and shadows the old one. The old slot stays alive.

```
i64 x = 5;
i64 x = 9;   // a new binding, not an overwrite
x            // 9
```

Three readings were possible. Updating the slot in place was the old behaviour, and it was the only sensible one while redeclaring was the only way to change a value. Erroring was rejected because retyping a declaration is a normal way to work at a prompt.

Shadowing wins for two reasons. The first is that it is the only option that survives a second type: `i64 x` and, later, `f64 x` cannot share storage, so an in-place update is not even expressible once types exist.

The second is that its apparent cost is not a cost. Code compiled earlier holds the **old** address and keeps reading the old slot after a redeclaration. That looks like a trap, but it is exactly lexical scoping, and it is what GHCi does: a function defined before a rebinding goes on seeing the binding it was defined against. Baking addresses into compiled code gives that behaviour for free rather than making it a bug to work around.

Nothing observable follows from this yet, because a compiled line is called once and thrown away. It starts to matter the moment functions exist, which is why it was worth settling first.

`sym_lookup` therefore scans backward, so the newest declaration of a name wins, and shadowed entries stay in the table because code still points at their slots.

8.4. The Last Value

BUILT

`it` is the value of the last expression.

The original plan called this `$?`, kept from bash. The requirement was that it be a real typed variable rather than bash's stringly-typed magic. GHCi, which vsh already borrows its REPL feel from, calls this `it`, and taking that name satisfies the requirement better than keeping `$?` would: `it` is an ordinary entry in the symbol table, so no special case reaches the lexer, the parser, or codegen. Only the REPL's assignment to it is special.

It is declared at startup like any other name, and it can be read, assigned, and even redeclared, all without any of those paths knowing it is special. The prompt looks its slot up rather than caching it, because redeclaring `it` shadows it with a new slot and the prompt should write the binding the next line will read.

Bash's other magic is answered the same way. `$1` and `$2` become ordinary typed function parameters, once functions exist, because a command in vsh is just a function.

9. Error Handling

9.1. At the Prompt

BUILT

A failure prints and the session continues. There is no exception, no unwinding, and no hidden control flow. Every stage reports through a return value, and the REPL prints the message and takes the next line.

This is already load-bearing: a bad line never damages the session, and the symbol table and every prior value survive it.

9.2. Silent Wrongness Is The Enemy

DECIDED

The one thing vsh must not do is confidently answer a different question than the one asked. This is the failure bash normalises, with its ignored exit codes and its unquoted expansions.

Three cases exist so far and were treated as bugs, not curiosities:

- An integer literal too large for `i64` is a diagnostic, not a wrap.

- A line longer than the input buffer is rejected. It used to be silently truncated, which meant a 6001-byte expression printed a confident, wrong `2048`. Truncation is now detected and the rest of the line discarded.
- Division by zero reports, rather than quietly yielding the zero that arm64 hands back. See below.

9.3. Division By Zero

BUILT

`5 / 0` reports an error and the session lives.

```
vsh> 5 / 0
error: division by zero
vsh> 42
42
```

arm64 `sdiv` yields zero for a zero divisor and never traps, so without a check this was a confident wrong answer, which is the one thing vsh must not do.

Three answers were possible. Trapping was rejected: it is loud and cheap, but a shell must not die because you typed `5 / 0`. Returning a tagged value is the brief's soft-failure design and is the eventual destination, but it needs a type system to express and so is not affordable yet. A runtime check is what is affordable now, and it is a real answer rather than a placeholder.

9.3.1. The Trap Channel

The generated code takes a `trap` slot, an ordinary `i64` in the session arena whose address is baked into the code like any variable's. Division emits a branch around itself:

```
    cbz   x1, fail        // divisor is zero
    sdiv  x0, x0, x1
    b     done
fail:
    mov   x2, #TrapKind_DivideByZero
    mov   x3, #&trap
    str   x2, [x3]
    mov   x0, #0
done:
```

The prompt clears the slot, calls, and reads it back. A non-zero slot means the value is meaningless and an error is printed instead.

The trap path **falls through** rather than returning. Returning from the middle of an expression would strand whatever operands the surrounding expression had already pushed, and balancing that would need a prologue that nothing else here wants. Carrying on with a zero costs nothing, because the caller throws the value away.

This is the first thing in vsh that needed branches, so the backend grew `cbz`, `b`, and the patching that fills a branch's offset in once its target is known. Control flow needs all of that anyway.

9.3.2. Consequences

A declaration that traps leaves its name declared, holding zero, because `check.c` created the name before any code ran. `i64 z = 5 / 0;` reports the error, and `z` afterwards is 0 rather than absent. That is a wart. Fixing it means deferring the declaration until the line has run, which is a larger change than it looks.

The check is genuinely at runtime, not folded at compile time. A zero divisor that only appears at runtime, as in `1000 / (5 - 5)`, is caught the same way.

9.4. Fallible Operations, Generally

DECIDED

The intent, not yet built: a function that can fail returns a tagged value rather than setting a global. There is no `errno`. At the prompt a failure prints inline and the session lives. The compiler should ideally warn when a fallible result is used without being checked.

OPEN Whether that is a hard error or a warning is open. It decides how the REPL actually feels, which is not something to settle on paper.

9.5. Exit

DECIDED for the shape. OPEN for the semantics.

Two mechanisms, deliberately named differently, because bash conflating them confuses people regularly.

`exit(code)`	Ends the current command or expression. Not the session. Should propagate up a pipe roughly the way process exit does in bash.
`exit_shell()`	Ends the interactive session. Deliberately harder to type by accident.

OPEN What `exit()` means inside an in-process pipe stage is the open question. Such a stage is a function call, not a process, so there is nothing to exit. Whether it unwinds only that function, or emulates real process-exit semantics despite no process existing, has to be settled before the pipe dispatcher is written.

Nothing here is built. Ctrl-D ends the session today, and that is all.

10. Processes and Pipes

DECIDED. None of this exists.

10.1. Hybrid Pipes

`|` means two different things depending on what is on each side.

vsh to vsh	An in-process function call. Typed, fast, no serialisation, no `fork`, no `exec`.
vsh to a binary	A real pipe, through an explicit builtin such as `sh("grep foo")`, doing `fork`, `exec`, `pipe(2)`, and `dup2` underneath.

The split keeps the fast typed path fast without losing the ability to call any Unix tool. A pure in-process design would lose the second; a pure process design would lose the first.

Routing to a binary is explicit rather than inferred. Guessing which one the user meant is the kind of magic vsh is trying not to have.

10.2. Environment

DECIDED

Environment variables have to exist, because child processes expect them. Inside vsh they should be a typed table, with an explicit builtin to export a value out to a child, rather than bash's untyped ambiently-inherited string soup.

11. Files

BUILT

```
vsh script.vsh    # runs it, prints nothing, exits 0
vsh               # the prompt
```

A file is compiled and run as **one unit**, not a line at a time. Nothing about the language had to change to allow it. Newlines were already only whitespace, the grammar already spanned them, and the semicolon rule already meant a sequence of statements produces no output. A file is simply the prompt's grammar without the trailing expression.

The whole file is read onto the scratch arena with `os_file_read` and handed to the same evaluator a line goes to. Ten thousand statements compile and run in about twelve milliseconds, and cost one stack frame, because statements chain and both walkers loop.

A file gets an exit status, because something other than a person is reading it: 0 when it ran, 1 when it could not be read or something in it failed, 2 for misuse. The prompt has no such thing, and no single line can end it.

OPEN A file is one compilation unit, so an error anywhere means none of it runs. That is the right default for a script. It is not obviously right for a `load` at an interactive prompt, where running the good half might be wanted. That question can wait for `load`, which needs strings first.

12. Implementation Map

BUILT

Where to look. Data flows top to bottom.

`src/base/`	Vendored from c-project-template at `67630fb`. Arenas, strings, arrays, logging, and the platform layer. vsh adds architecture detection and the executable-memory pair.
`src/lex/`	Bytes to tokens. Allocates nothing; tokens borrow the source.
`src/parse/`	Tokens to a tree, on a caller's arena. Recursive descent.
`src/sym/`	The session symbol table. Owns names and slots. Hands out slot pointers only.
`src/check/`	Name resolution. Fills each node's slot so codegen cannot fail on a name. Where the type checker goes.
`src/jit/`	Tree to arm64, and the executable code arena. arm64 only; `#error`s elsewhere.
`src/repl/`	Owns the session: both arenas, the symbol table, the code arena, and `it`. Drives the pipeline.

The whole program is one translation unit. `src/main.c` is the only file the compiler sees; everything else arrives through `src/unity_build.c`, in dependency order.

`STYLE.md` is the authority on conventions and is not optional reading.

13. Open Questions, Collected

The things that are genuinely undecided, gathered from above. These are where the design work actually is.

1. **Unchecked fallible results.** Hard error or warning?
2. **Tagged fallible values**, which need a type system, and which should eventually replace the trap channel that division by zero introduced.
3. **A declaration that traps** still declares its name, holding zero.
4. **`exit()` inside an in-process pipe stage.** There is no process to exit.
5. **Compile granularity.** One line as one function, or explicit blocks?
6. **Reclaiming code space.** Compiled bytes currently live for the whole session.
7. **Implicit conversion**, once there is more than one type.

The brief these came from was explicit that several should be worked out **through** prototyping rather than settled on paper. That still holds. This document records the tension, not a verdict.